

Python Programming

Unit 03 – Lecture 01 Notes

File Handling Fundamentals and Directories

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	1
2	Core Concepts	2
2.1	The <code>open()</code> Function	2
2.2	File Modes (Read/Write/Append)	2
2.3	Reading Data	2
2.4	Writing Data	3
2.5	Why <code>with</code> is Better Than Manual Close	3
2.6	File Pointer: <code>tell()</code> and <code>seek()</code>	3
2.7	Working with Directories (Pathlib)	3
3	Demo Walkthrough	4
3.1	Demo 1: Read/Write + Simple Stats	4
3.2	Demo 2: Directory Walk	4
4	Interactive Checkpoints (with Solutions)	4
5	Practice Exercises (with Solutions)	5
6	Exit Question (with Solution)	5

1 Lecture Overview

Many programs become truly useful only when they can **store and load** data. File handling allows you to:

- save results (reports, logs, grades, summaries),
- reuse data later (datasets, configuration),
- and share data between programs.

This lecture covers file opening modes, reading/writing patterns, the `with` statement, and basic directory handling using `pathlib`.

2 Core Concepts

2.1 The open() Function

The function `open(path, mode, encoding=...)` returns a file object. You can use it to read or write.

Best practice: use `with open(...)` so the file is always closed:

```
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)
```

2.2 File Modes (Read/Write/Append)

Common modes (text mode):

Mode	Meaning
r	read (file must exist)
w	write (overwrite if exists, create if not)
a	append (write at end, create if not)
x	create new (error if file exists)
t	text mode (default)
b	binary mode (images, PDFs, etc.)
+	update mode (read + write)

Important behavior:

- "w" truncates the file (clears old content). Use carefully.
- "a" keeps old content and adds new lines at the end.
- "r" fails if the file does not exist (`FileNotFoundError`).

2.3 Reading Data

Reading the whole file

```
with open("names.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

Good for small files. Not recommended for very large files.

Reading line by line (recommended)

```
with open("names.txt", "r", encoding="utf-8") as f:
    for line in f:
        name = line.strip()
        print(name)
```

`strip()` removes the newline character "
n" and extra spaces.

2.4 Writing Data

To write a line, use `write()` and include a newline `"\n"`:

```
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("Alice\n")
    f.write("Bob\n")
```

To write many lines at once, use `writelines()`:

```
lines = ["A\n", "B\n", "C\n"]
with open("out.txt", "w", encoding="utf-8") as f:
    f.writelines(lines)
```

2.5 Why with is Better Than Manual Close

Without `with`, you must remember `f.close()`:

```
f = open("data.txt", "r")
try:
    print(f.read())
finally:
    f.close()
```

`with` automatically does this for you. This matters on Windows because open files can remain *locked* if not closed properly.

2.6 File Pointer: `tell()` and `seek()`

Files have a current reading position.

```
with open("data.txt", "r") as f:
    print("pos =", f.tell())
    first = f.readline()
    print("after readline pos =", f.tell())
    f.seek(0) # move back to start
    again = f.readline()
```

2.7 Working with Directories (Pathlib)

The `pathlib` module gives an object-oriented way to handle paths.

```
from pathlib import Path

root = Path(".")
print("Current folder:", root.resolve())

for p in root.iterdir():
    print(p.name, "dir" if p.is_dir() else "file")
```

Common useful operations:

- Create a directory: `Path("data").mkdir(exist_ok=True)`

- Join paths: `root / "data" / "names.txt"`
- Recursive search: `root.rglob("*.py")`

3 Demo Walkthrough

3.1 Demo 1: Read/Write + Simple Stats

File: `demo/file_read_write_demo.py`

What it does:

- Creates a file in `data/` and writes a few names.
- Reads the file back.
- Computes:
 - number of names,
 - longest name,
 - count of names that start with a vowel.

3.2 Demo 2: Directory Walk

File: `demo/directory_walk_demo.py`

What it does:

- Prints a small “tree” of files under the lecture folder.
- Demonstrates `Path.rglob(...)` and basic path operations.

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: difference between modes `"w"` and `"a"`?

Answer:

- `"w"` overwrites (truncates) the file.
- `"a"` appends to the end of the file without deleting old content.

Checkpoint 2 Solution

Question: what happens if you open a missing file using `"r"`?

Answer: Python raises `FileNotFoundError`.

5 Practice Exercises (with Solutions)

Exercise 1: Count Lines

Task: Read a text file and print number of lines.

Solution:

```
path = input("File path: ").strip()
count = 0
with open(path, "r", encoding="utf-8") as f:
    for _ in f:
        count += 1
print("Line count =", count)
```

Exercise 2: Copy a File

Task: Copy text from one file to another.

Solution:

```
src = input("Source file: ").strip()
dst = input("Destination file: ").strip()
with open(src, "r", encoding="utf-8") as f_in:
    with open(dst, "w", encoding="utf-8") as f_out:
        for line in f_in:
            f_out.write(line)
print("Copied.")
```

Exercise 3: List Python Files

Task: Print all .py files inside the current directory and its subfolders.

Solution:

```
from pathlib import Path
root = Path(".")
for p in root.rglob("*.py"):
    print(p)
```

6 Exit Question (with Solution)

Question: Read all lines from "names.txt" using with and print without newlines.

Solution:

```
with open("names.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Open files using correct access modes (read/write/append)
- Read and write text data using common file methods
- Use the `with` statement to manage file resources safely
- Work with directories using `pathlib` / `os`
- Apply file handling to simple real-world tasks (logs, reports, data)

Why File Handling?

- Data should **persist** after the program ends
- Files help in: logs, configuration, reports, datasets
- Working with files is a foundation for:
 - CSV processing
 - data analysis pipelines
 - web and GUI apps that store data

The Basic Pattern

```
with open("data.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

- `open()` returns a file object
- Always close files (the `with` block does it automatically)

File Modes (Text)

Mode	Meaning
r	read (file must exist)
w	write (overwrite / create new)
a	append (write at end / create new)
x	exclusive create (fail if exists)
t	text mode (default)
b	binary mode (images, PDFs, etc.)
+	update (read + write)

Reading from a File

Common methods:

- `read()` → whole file as one string
- `readline()` → one line
- `readlines()` → list of lines
- Best practice: iterate line-by-line (memory friendly)

```
with open("names.txt", "r") as f:
    for line in f:
        name = line.strip()
        print(name)
```

Writing to a File

- `write()` writes a string (you manage newlines)
- `writelines()` writes a list of strings

```
lines = ["Alice\n", "Bob\n", "Charlie\n"]
with open("names.txt", "w") as f:
    f.writelines(lines)
```

The with Statement

- A **context manager** that guarantees closing the file
- Even if an exception happens, the file is closed properly
- Reduces resource leaks and locking issues

File Pointer: tell() and seek()

- `tell()` gives current position
- `seek(pos)` moves to a position
- Useful when you need to re-read or skip parts

```
with open("data.txt", "r") as f:
    print(f.tell())
    first = f.readline()
    f.seek(0)
    again = f.readline()
```

Working with Directories

- Use `pathlib.Path` for readable path handling
- Common tasks:
 - list files in a folder
 - create directories
 - join paths safely

```
from pathlib import Path
p = Path(".")
for f in p.iterdir():
    print(f)
```

Demo: Read/Write + Directory Listing

- `demo/file_read_write_demo.py`
 - writes a small file in `data/`
 - reads it back and computes simple stats
- `demo/directory_walk_demo.py`
 - lists files and folders under the lecture directory

Checkpoint 1

Question: What is the difference between modes `"w"` and `"a"`?

Write one example for each.

Checkpoint 2

Question: What happens if you open a non-existing file using `"r"`?

Think-Pair-Share

Discuss:

- What should be stored in files vs kept only in memory?
- Give 2 examples for each category.

Key Takeaways

- Choose correct file mode: `r/w/a/x` (text vs binary)
- Prefer `with open(...)` to auto-close files safely
- Read line-by-line for large files
- Use `pathlib` for clean directory and path handling

Exit Question

Write a code snippet to read all lines from `"names.txt"` safely using `with open(...)` and print them without newline characters.